

Based on the provided sources and the specific finding details, here is the comprehensive analysis and structured report for the security issue regarding Fee-on-Transfer tokens in the refund logic.

Summary

The use of Fee-on-Transfer (FoT) tokens creates an insolvency state within the createBid function. Because the contract tracks the *nominal* bid amount rather than the *received* amount, it attempts to refund previous bidders more tokens than it actually holds, causing valid subsequent bids to revert and effectively jamming the auction.

Finding Description

The **Trader Bros Auction** protocol advertises "Flexible Payments" with support for "any ERC20 token" 1, 2. However, the current implementation of the bidding logic creates a discrepancy between internal accounting and actual token balances when handling tokens with transfer taxes.

The Flaw: In createBid, the contract performs two critical actions:

1. **Ingress:** It transfers tokens from the new bidder to itself:
paymentToken.safeTransferFrom(msg.sender, address(this), amount).
2. **Refund:** If there is an existing bidder, it refunds them:
paymentToken.safeTransfer(_auction.bidder, _auction.amount).

The Failure Path: If the configured payment token has a transfer fee (e.g., 1%):

1. **User A** bids 100 tokens. The contract receives 99 tokens (due to fee), but updates _auction.amount to 100.
2. **User B** attempts to outbid User A.
3. The contract attempts to refund **User A** the recorded 100 tokens.
4. Since the contract only holds 99 tokens (assuming no excess balance from other sources), the safeTransfer to User A reverts.

This breaks the **Liveness** guarantee of the auction. Once a user bids with a FoT token, they become the permanent "King of the Hill" because no one else can successfully execute the function to displace them.

Impact Explanation

Severity: High (Functional Denial of Service) This vulnerability allows for the permanent stalling of an active auction.

1. **Auction Deadlock:** Valid higher bids are blocked because the contract cannot satisfy the refund condition for the previous bidder.
2. **Revenue Loss:** The protocol fails to capture higher bids, and if the auction cannot resolve, the "Auto-Settlement" 1 may also fail or result in incorrect payouts.
3. **Malicious Griefing:** An attacker could intentionally bid with a FoT token (if whitelisted) to freeze the auction at a low price, preventing legitimate competition.

Likelihood Explanation

Likelihood: Medium The protocol explicitly supports "Token Switching" between auctions 1 and allows "any ERC20 token" 2.

- The system allows the owner to updatePaymentParams 3.
- If the owner unknowingly sets a deflationary token (like PAXG or certain meme coins) as the payment currency, this issue triggers immediately upon the first bid.
- Given the complexity of ERC20 compliance, accidentally enabling a non-standard token is a common operational error.

Proof of Concept

The following test scenario demonstrates how the auction jams when a new bidder tries to beat a bid made with a FoT token.

```
// Assumptions:
// - Payment token has a 1% transfer fee.
// - Auction is active.

function test_FoT_Refund_DoS() public {
    // 1. Setup: Payment token is Fee-On-Transfer (1% fee)
    MockFeeToken foToken = new MockFeeToken("FeeToken", "FOT", 1);
    auction.setPaymentToken(address(foToken));

    // 2. Alice bids 100
    // Transfer: Alice sends 100 -> Auction receives 99.
    // State: highestBid = 100.
    vm.startPrank(alice);
    foToken.approve(address(auction), 100);
    auction.createBid(100);
    vm.stopPrank();

    // 3. Bob tries to bid 110 (outbid Alice)
    // Logic requires refunding Alice 100.
    // Contract balance: 99.
    vm.startPrank(bob);
    foToken.approve(address(auction), 110);

    // 4. This call REVERTS because auction lacks funds to refund
    Alice
    vm.expectRevert("ERC20: transfer amount exceeds balance");
    auction.createBid(110);
    vm.stopPrank();

    // Result: Alice wins with a low bid, Bob is censored.
}
```

Recommendation

To safely support "any ERC20 token" as specified in the architecture 2, the contract must adopt a **"Balance-Delta"** accounting method. The bid amount recorded should reflect the actual tokens received, not the amount sent.

Corrected Code Snippet:

```
function createBid(uint256 amount) external nonReentrant {
    // ... existing checks ...

    // 1. Snapshot balance before transfer
    uint256 balanceBefore =
IERC20(paymentToken).balanceOf(address(this));

    // 2. Perform transfer from new bidder
    IERC20(paymentToken).safeTransferFrom(msg.sender, address(this),
amount);

    // 3. Calculate ACTUAL received amount
    uint256 balanceAfter =
IERC20(paymentToken).balanceOf(address(this));
    uint256 actualReceived = balanceAfter - balanceBefore;

    // 4. Validate bid against the ACTUAL received amount
    require(actualReceived >= currentBid + minIncrement, "Bid too low
after fees");

    // 5. Refund previous bidder (if exists)
    if (_auction.bidder != address(0)) {
        // Refund the recorded amount (which was calculated via delta
previously)
        IERC20(paymentToken).safeTransfer(_auction.bidder,
_auction.amount);
    }

    // 6. Update state with ACTUAL amount
    _auction.amount = actualReceived;
    _auction.bidder = msg.sender;

    emit BidCreated(msg.sender, actualReceived);
}
```

ELI5 Analogy: The "Exact Change" Coat Check

Imagine the Auction is a coat check where you can buy the "Best Spot" by paying cash. The rule is: if someone pays more than you, the attendant gives you your money back, and the new person takes the spot.

1. **The Situation:** The Coat Check accepts "Sticky Dollar Bills" (Fee-on-Transfer tokens). Every time you hand a Sticky Bill to someone, a small corner tears off and disappears.
2. **The First Guy:** Alice hands the attendant a \$100 Sticky Bill. The attendant puts it in the register, but because it's sticky, a piece tears off. The register actually only has **\$99** inside. But the attendant writes "Alice: \$100" on the clipboard.
3. **The Problem:** Bob comes along and offers \$110. The attendant says, "Okay, let me give Alice back her \$100."
4. **The Jam:** The attendant looks in the register. There is only \$99. He cannot give Alice back the \$100 he promised. He panics and freezes. Bob cannot buy the spot, and Alice keeps it effectively for free (or at a discount).

The Fix: The attendant should count the money *after* it goes into the register (\$99), write "\$99" on the clipboard, so he knows exactly how much to give back later.